MyCodeSchool

L1.5 — Deploy pe un server AWS

Pista DevOps, lecția 1, pasul 5 din 5

Obții aceleași șase servicii de la pasul 4 rulând pe un server Ubuntu din AWS, pornite cu același `docker-compose.yaml` și deschise din browser pe IP-ul serverului; la final oprești serverul ca să nu plătești. Lucrezi în același terminal din VS Code, profilul **„WSL: Ubuntu"**, în rădăcina repo-ului (lângă `docker-compose.yaml`). Peste tot, *IP-UL-SERVERULUI* = adresa de la punctul 1.

1. Serverul EC2, din consola AWS

În `https://console.aws.amazon.com/ec2`, sus-dreapta regiunea **Europe (Frankfurt) eu-central-1** → **Launch instance**:

• **Name**: `devops-initiere` • **AMI**: `Ubuntu Server 24.04 LTS`, `64-bit (x86)` • **Instance type**: `t3.small`
• **Key pair** → **Create new key pair** → nume `devops-initiere`, `RSA`, `.pem` → **Create**: se descarcă `devops-initiere.pem` în `Downloads`
• **Network settings** → **Edit**: la regula `ssh`, port `22`, **Source type** = `My IP`; **Add security group rule** de cinci ori: `Custom TCP`, porturile `8101`, `8102`, `8103`, `8106`, `8110`, fiecare cu `My IP`
• **Configure storage**: `20` GiB `gp3` → **Launch instance**.

Trebuie să vezi: la **Instances**, `devops-initiere` cu `Running` și `3/3 checks passed`; copiezi **Public IPv4 address** în `notite-L1.txt`.

2. Cheia .pem în Ubuntu

```
ls /mnt/c/Users/
```

Trebuie să vezi: numele tău de utilizator din Windows, lângă `Public`; îl pui în comanda de mai jos.

```
mkdir -p ~/.ssh
cp /mnt/c/Users/UTILIZATORUL-WINDOWS/Downloads/devops-initiere.pem ~/.ssh/
chmod 400 ~/.ssh/devops-initiere.pem
ls -l ~/.ssh/devops-initiere.pem
```

Trebuie să vezi: `-r-------- 1 …`. Fără `chmod 400`, la pasul 3 primești `WARNING: UNPROTECTED PRIVATE KEY FILE!`.

3. Intri pe server

```
ssh -i ~/.ssh/devops-initiere.pem ubuntu@IP-UL-SERVERULUI
```

Trebuie să vezi: prima dată `Are you sure you want to continue connecting (yes/no/[fingerprint])?` → scrii `yes`; apoi promptul `ubuntu@ip-172-31-…:~$`. `Connection timed out` = regula pentru `22` nu are IP-ul tău de acum; `Permission denied (publickey)` = altă cheie sau alt utilizator decât `ubuntu`.

4. Docker Engine + plugin-ul compose, pe server

Scriptul oficial de instalare de pe `get.docker.com` pune Docker Engine și plugin-ul compose:

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker ubuntu
exit
```

Trebuie să vezi: la final `logout` și `Connection to … closed.`: ieși ca să intre în vigoare grupul `docker`.

```
ssh -i ~/.ssh/devops-initiere.pem ubuntu@IP-UL-SERVERULUI
docker --version
docker compose version
```

Trebuie să vezi: `Docker version 29.…` și `Docker Compose version v5.…`, fără `sudo`. `permission denied … docker.sock` = n-ai ieșit și n-ai intrat din nou.

5. Memorie în plus (swap), pe server

`t3.small` are 2 GB pentru șase containere; swap-ul e rezerva pe disc care nu lasă o aplicație să se oprească singură când memoria se umple.

```
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
free -h
```

Trebuie să vezi: rândul `Swap:` cu `2.0Gi` în coloana `total`.

6. docker-compose.yaml pe server

Pe server, folderul cu același nume ca repo-ul, ca să păstrezi numele containerelor:

```
mkdir ~/devops-initiere
```

În VS Code deschizi un al doilea terminal (**+** → **WSL: Ubuntu**), în rădăcina repo-ului, și trimiți fișierul de la pasul 4:

```
scp -i ~/.ssh/devops-initiere.pem docker-compose.yaml ubuntu@IP-UL-SERVERULUI:~/devops-initiere/
```

Trebuie să vezi: `docker-compose.yaml 100%`. Te întorci în primul terminal, cel cu `ubuntu@ip-172-31-…`.

7. Pornești, pe server

```
cd ~/devops-initiere
docker compose up -d
```

Trebuie să vezi: `Pulled` la cele șase imagini (inclusiv `CONTUL-TAU/devops-online-school-api:1.1`), `Container devops-initiere-mysql-1 Healthy` și cinci `Started`. `pull access denied` = imaginea e `Private` pe Docker Hub sau numele contului e greșit.

```
docker compose ps
```

Trebuie să vezi: șase rânduri `Up`, `mysql (healthy)`, cu `0.0.0.0:8101->8080/tcp`, `8102`, `8103`, `8106` și `8110` la fel.

```
docker compose logs | grep "Now listening"
```

Trebuie să vezi: cinci rânduri `Now listening on: http://[::]:8080`, câte unul per aplicație. Pe server durează mai mult; dacă lipsește unul, aștepți 30 de secunde și repeți.

8. Swagger din browser-ul tău

În browser-ul din Windows deschizi pe rând `http://IP-UL-SERVERULUI:8101/swagger`, apoi la fel pe `8102`, `8103`, `8106` și `8110`.

Trebuie să vezi: titlurile `mock-car-source-api`, `OneModelApi`, `OneToManyApi`, `OnlineShopApi` și `Online School Cloud API`. Pagina care se încarcă la nesfârșit = portul nu e în security group.

Pe `8103`: `GET /api/users` → **Try it out** → **Execute**.

Trebuie să vezi: `Code 200` și primul utilizator cu `"name": "John Doe"`: datele vin din MySQL-ul de pe server.

9. Oprești serverul

```
exit
```

Trebuie să vezi: `Connection to … closed.`

În consola EC2: bifezi `devops-initiere` → **Instance state** → **Stop instance** → **Stop**.

Trebuie să vezi: `Stopping`, apoi `Stopped`. Oprit, plătești doar discul; la următorul **Start**, IP-ul public e altul.

Gata când…

Ai deschis cele cinci Swagger-e pe `http://IP-UL-SERVERULUI`, cu porturile `8101`, `8102`, `8103`, `8106` și `8110`, iar instanța `devops-initiere` e `Stopped`. Ai terminat lecția 1.

În `notite-L1.txt` adaugi un rând:

```
L1.5: IP public: ÎNLOCUIEȘTE-CU-IP-UL | Swap total în free -h: ÎNLOCUIEȘTE-CU-valoarea |
Instance state la final: ÎNLOCUIEȘTE-CU-starea
```